CAPSTONE ARCHITECTURE PLAN

Multi-Agent E-commerce Support Agent

A detailed, phased implementation plan aligned to the capstone rubric, with layered architecture, specialist contracts, safety controls, evaluation design, and submission evidence.

Recommended architectural pattern
Central supervisor/orchestrator + bounded specialist agents + shared domain services + independent policy enforcement + observable evaluation.

Planning principle
Build and measure the baseline first. Add capabilities incrementally, preserve evidence at each stage, and keep policy, authorization, and action execution outside free-form model control.

Prepared as the implementation blueprint for the nine required capstone phases.

Executive summary

This plan turns the capstone requirements into an implementation blueprint for a realistic e-commerce support workflow. The final system uses a central orchestrator to route work to order, returns, logistics, product, and escalation specialists. Shared services provide retrieval, structured mock-data access, session memory, feedback handling, and observability.

The system is intentionally safety-first: the LLM can interpret a request, propose a plan, and draft an answer, but deterministic services enforce policy, authorization, confirmation, and action-state transitions. All write actions are simulated for the capstone.

The required evolution is preserved: rule-based baseline → LLM → RAG → tools → planning and memory → adaptation → deployment readiness → evaluation and engineering review.

1. Architecture review and key decisions

Area	Decision / recommendation
Business scope	Focus on Tier-1 e-commerce support: order status, returns, shipping, product questions, and escalation.
Baseline vs. target	Demonstrate the single-agent rule-based baseline and subsequent stages before introducing the final multi-agent architecture.
Authority boundaries	LLM interprets and drafts; deterministic policy and authorization services decide eligibility and action permission.
Knowledge sources	Policy/catalog/FAQ use RAG; order status and shipment events come from structured mock services.
Transactions	Simulated writes only; require exact action confirmation, authorization, and result verification.
Feedback	Use bounded, auditable configuration changes; never let raw feedback rewrite policy or disable safeguards.
Framework strategy	Start with plain Python interfaces; add an orchestration framework only if it materially improves tracing, routing, or maintainability.
Monitoring	Begin with structured logs and trace IDs; add one tracing backend only if available and useful.

Design principles

- **Explicit boundaries:** interaction, orchestration, specialist agents, domain services, safety/policy, and operations/evaluation.
- **Typed contracts:** validate agent results and tool inputs/outputs; treat model-generated action proposals as data, not permission.
- **Evidence-first responses:** answer from retrieved sources or verified records; clarify, abstain, or escalate when evidence is missing.
- **Bounded execution:** cap delegation, tool calls, retries, and planning steps.
- **Reproducibility:** version prompts, datasets, knowledge snapshots, configuration, and evaluation outputs.
- **Safety as a feature:** refusal, uncertainty, escalation, confirmation, and PII-safe logging are tested behaviors.

2. Problem framing and success definition

Primary persona: an e-commerce customer seeking accurate answers about an order, shipment, return, refund policy, or product. Secondary persona: a Tier-1 support analyst who needs a concise, evidence-backed handoff for exceptions.

Problem statement

Customers need timely and consistent support, but policy knowledge, order state, shipping events, and product information are different kinds of data. A useful support agent must retrieve the right policy, verify structured records, explain what is known, and route exceptions without inventing facts or taking unauthorized actions.

Scope and assumptions

- Initial deployment is a local CLI; an API can be added after the workflow stabilizes.
- Use synthetic policies, product records, orders, and tracking events; do not connect to real customer systems.
- All transactional actions are simulated and auditable.
- The LLM provider is configurable through an interface; secrets remain outside source control.
- Policy thresholds and retention periods are explicit configuration, not guessed by the model.
- Human escalation is represented by a mock queue and a verifiable ticket result.

Example user questions

1. “I received my shoes 12 days ago. Can I return them?”
2. “My package is delayed. What is the latest tracking event, and can I request a refund?”
3. “Cancel my order.”
4. “I want a refund outside the return window, and the product caused an injury.”
5. “Can you make the answer shorter? Also, what does that mean for my order?”

Success criteria

- Supported questions receive accurate answers grounded in the correct source or verified record.
- Unsupported or conflicting evidence leads to clarification, uncertainty, or escalation—not invented facts.
- No prohibited or unconfirmed simulated action executes.
- Prompt variants are compared on the same frozen dataset.
- Retrieval, tools, memory, adaptation, safety, failure recovery, and deployment each have concrete evidence.
- The application can be started and evaluated using documented commands in a clean environment.

Known edge cases

- Missing/unknown order IDs; incomplete or stale tracking events.
- Conflicting policy versions; product-category exclusions; boundary dates.
- Malformed tool arguments, nonexistent tools, timeouts, duplicate action attempts.
- Ambiguous “yes” confirmation; changed action parameters; expired pending action.
- Prompt injection in user input or retrieved documents; PII in inputs and logs.
- Memory conflicts with a current verified record; reset/deletion behavior.

3. Target architecture and layer boundaries

Use a supervisor–specialist pattern with deterministic control flow and bounded delegation. The orchestrator owns routing, plan execution, state transitions, budgets, and final assembly. Specialists interpret domain requests and call only approved tools. Shared services provide retrieval, structured records, policy calculations, memory, feedback, and observability.

Logical architecture

Layer	Responsibilities	Must not own
1. Interaction	CLI/API, request validation, session identity, response presentation	Business rules or direct data writes
2. Application/control	Safety gateway, orchestrator, workflow state, routing, policy gates	Duplicated policy logic hidden in each agent
3. Specialist agents	Order, returns, logistics, product, escalation interpretation	Unrestricted tools or policy overrides
4. Shared domain services	RAG, mock records, eligibility calculation, memory, feedback	Conversational control of the whole workflow
5. Cross-cutting operations	Provider adapters, configuration, logs, traces, metrics, evaluation	Raw sensitive conversation logging

Specialist responsibilities

Agent	Owns	Output / boundary
Order Agent	Order state, dispatch state, cancellation eligibility lookup	Verified record and permitted next steps; no independent cancellation
Returns Agent	Return-window and category-policy interpretation	eligible / ineligible / unknown / human review; no independent refund
Logistics Agent	Carrier events, delays, delivery estimate interpretation	Latest known event and timestamp; distinguish estimate from guarantee
Product Agent	Catalog specs, sizing, compatibility, care	Catalog-grounded answer; disclose missing specifications
Escalation Agent	Handoff packet and queue selection	Sanitized summary, reason, priority, sources; success only after tool confirmation

End-to-end lifecycle

1. Validate request; redact or minimize sensitive fields before logging.
2. Apply safety and risk classification. Refuse prohibited requests or route high-risk cases.
3. Load only permitted session context and classify intent.
4. Create a bounded plan and route to the appropriate specialist(s).
5. Retrieve policy evidence or call an approved read-only tool.
6. Validate evidence and tool results; clarify or escalate if insufficient.
7. Apply deterministic policy and authorization checks.
8. For an action, validate exact parameters and require a matching explicit confirmation.
9. Execute only an approved simulated write; verify the returned state.
10. Compose a response with source references, record outcome, and update only permitted session state.

Required invariants

- No unsupported policy claim is stated as fact.
- A confirmation is bound to session, action ID, action type, exact parameters, and expiry.
- A specialist cannot grant itself permissions through generated tool calls.
- Tool failure is never silently treated as success.
- Memory cannot override current policy or verified records.
- Every request has an auditable terminal outcome; retries and delegation have finite budgets.
- Sensitive values are minimized and redacted before logs are written.

4. Domain services, tools, and typed contracts

Proposed tools

Tool	Access	Purpose
search_policy	Read-only	Retrieve authoritative policy passages and citations
lookup_order	Read-only	Look up synthetic order records
get_tracking_events	Read-only	Retrieve synthetic carrier events
get_product_details	Read-only	Retrieve product specifications
check_return_eligibility	Read-only calculation	Apply configured rules to verified inputs
create_return_request	Simulated write	Demonstrate validation, confirmation, and state transition
create_escalation	Controlled mock write	Create a handoff and return a verified ticket ID
search_web	External read	Optional general context; never replace authoritative retailer policy

Every tool requires a typed input schema, validated output schema, declared access category, timeout, error contract, unique execution ID, permission check, and retry policy. The LLM must never execute arbitrary Python, shell commands, imports, or filesystem access.

Common agent result contract

```
class EvidenceRef(BaseModel): source_id: str section: str | None = None record_version: str | None = None class AgentResult(BaseModel): agent_name: str status: Literal[ "success", "needs_clarification", "insufficient_evidence", "tool_error", "escalation_required" ] summary: str evidence: list[EvidenceRef] = [] proposed_action: str | None = None escalation_reason: str | None = None
```

Contract rules

- Results are data, never executable code.
- Evidence IDs must resolve to retrieved documents or validated records.
- The orchestrator rejects malformed outputs and unknown status values.
- Proposed actions are not authorizations; policy and action gates run independently.

5. Phase-by-phase implementation plan

Phase 1 — Understand the problem and define success

Tasks

- Define primary and secondary personas, support workflow, supported intents, and out-of-scope requests.
- Document inputs, outputs, constraints, assumptions, authoritative sources, and escalation conditions.
- Define measurable success criteria and known failure cases.
- Create 3–5 canonical user questions and freeze the first evaluation dataset.

Artifacts: problem_framing.md, requirements.md, acceptance_criteria.md, evaluation_cases.json.
Exit gate: scope, risk boundaries, assumptions, and measurable criteria are agreed before coding.

Phase 2 — Build the rule-based Python baseline

- Build a modular CLI using simple intent matching and deterministic templates.
- Use small synthetic policy/order datasets and log sanitized request IDs, intent, response, and latency.
- Demonstrate at least two limitations: paraphrase failure and multi-intent failure.
- Explain why deterministic rules alone do not generalize to varied customer language.

Artifacts: baseline_agent.py, baseline_run.log, baseline_limitations.md.
Exit gate: reproducible run plus logged successful and failing examples.

Phase 3 — LLM integration and prompt evaluation

- Add a provider-independent model-client interface and versioned prompts.
- Define typed outputs for intent, planning summaries, and response content.
- Compare three prompt strategies on the same frozen cases and controlled settings.
- Record actual outputs, citations, tool calls, latency, and safety outcomes.
- Select a default based on safety/grounding gates first, then usefulness, escalation quality, latency, and cost.

Artifacts: llm_agent.py, prompts/, prompt_comparison.csv, prompt_evaluation.md.
Exit gate: default prompt selected from recorded evidence.

Phase 4 — Add knowledge and retrieval

- Prepare synthetic, versioned policy, catalog, FAQ, warranty, and escalation documents.
- Extract text, preserve section metadata, chunk documents, generate embeddings, and build a local FAISS index.
- Retrieve top-k evidence with source IDs, versions, sections, and relevance scores.
- Compare LLM-only and RAG responses on answerable, ambiguous, conflicting, and unanswerable questions.
- Abstain or escalate when evidence is missing, stale, or contradictory.

Artifacts: ingestion.py, chunks.json, index snapshot, retrieval evaluation, citation examples.
Exit gate: retrieval quality measured separately from answer quality.

RAG implementation notes

Start with approximately 400–700 tokens per chunk and 50–100 tokens of overlap; tune these against a labeled retrieval set. Preserve policy section boundaries. Filter by document type/category where appropriate. Use a calibrated relevance threshold rather than an assumed universal cutoff.

Suggested synthetic corpus

- return_policy.md; refund_policy.md; shipping_policy.md; cancellation_policy.md
- warranty_policy.md; faq.md; escalation_guidelines.md
- product_catalog.json; sample_orders.json; sample_tracking_events.json

Keep policy documents separate from structured order/tracking records: policy says what is allowed; verified records establish the current state of a particular order.

Retrieval evaluation

Test	Expected result
Direct policy question	Relevant section appears in top-k
Paraphrased question	Relevant section remains retrievable
Product/category query	Applicable category restrictions are retrieved
Unsupported question	No fabricated supporting source; abstain/escalate
Conflicting versions	Conflict surfaced rather than arbitrarily resolved
Unrelated query	Irrelevant chunks do not justify a confident answer

Phase 5 — Tool usage and multi-agent orchestration

- Implement typed tool contracts and an allowlisted registry.
- Add the five specialist agents and central supervisor routing.
- Apply authorization and policy checks before every sensitive operation.
- Test correct routing, malformed arguments, nonexistent records/tools, timeouts, denied actions, and loops.
- Demonstrate at least one intentional failed tool call and safe recovery.

Artifacts: orchestrator.py, agents/, tools/, tool-call traces, negative-test results.
Exit gate: bounded multi-step workflow works and prohibited/failed calls are safe.

Phase 6 — Planning, memory, and context

- Use explicit plan steps with input/output definitions, retry limits, and completion criteria.
- Maintain session context with a session identifier and defined TTL.
- Optionally store explicit user-approved preferences; avoid persisting full transcripts by default.
- Define retention, reset, deletion, access boundaries, and conflict handling.
- Test follow-ups, context switching, contradictory information, and reset.

Artifacts: planner.py, memory.py, multi-turn transcripts, memory policy.
Exit gate: improved multi-turn behavior without stale memory overriding authoritative sources.

Phase 7 — Adaptive behavior

- Collect explicit feedback using controlled categories such as helpful/not helpful and too verbose.
- Store sanitized feedback events separately from operational logs.
- Adjust only approved parameters such as response style or retrieval preferences.
- Require minimum sample size, validation, versioning, regression tests, and rollback.
- Show before/after behavior on the same evaluation cases.

Artifacts: feedback collector, adaptation policy, feedback store, before/after comparison.
Exit gate: measurable and reversible behavior change; mandatory safety policies remain immutable.

Phase 8 — Deployment readiness

- Pin dependencies and document environment assumptions; provide .env.example without credentials.
- Add correlation IDs, structured logs, stage timings, error categories, and token usage where available.
- Handle timeouts, invalid model output, empty retrieval, and unavailable tools gracefully.
- Test clean-environment startup and measure latency/error rates over a repeatable workload.
- Scan logs for synthetic PII canaries and secrets.

Artifacts: README.md, pyproject.toml/requirements, sample traces, latency/error report, optional Dockerfile.
Exit gate: another person can run the app and understand runtime failures.

Phase 9 — Evaluation and engineering review

- Run the full frozen evaluation suite against the final build.
- Measure answer quality, grounding, retrieval, routing, policy compliance, escalation, latency, and errors.
- Perform root-cause analysis and demonstrate a defect before and after a fix.

- Review prompt injection, PII handling, authorization, refusals, and human handoff.
- Document limitations and a prioritized improvement roadmap.

Artifacts: evaluation harness, evaluation_report.md, regression results, safety evidence, engineering justification.
Exit gate: each rubric capability has concrete evidence; limitations are disclosed.

6. Prompt comparison experiment

The required method is a controlled comparison of 2–3 prompt variants using the same test set. Keep model version, evidence, tool availability, and decoding settings constant where possible. Version/hash each prompt and capture actual output.

Variant	Prompt strategy	Hypothesis / tradeoff
A — Basic	Polite assistant role prompt	Baseline; likely to guess, omit citations, or miss escalation
B — Structured	Intent, evidence, uncertainty, typed response schema	Better consistency; may still overstate weak evidence
C — Evidence/safety constrained	Grounding, source citations, refusal/escalation, untrusted-data handling, action limits	Likely safer; may increase clarification/escalation

Required comparison table template

Test case	Prompt	Actual output	Improved / worsened
Known return window	A / B / C	Capture actual output	Correctness, structure, citations
Missing policy	A / B / C	Capture actual output	Guessing vs. safe abstention
Prohibited request	A / B / C	Capture actual output	Unsafe compliance vs. refusal
Conflicting sources	A / B / C	Capture actual output	Conflict recognition and escalation

These rows are an experiment template, not results. Run every prompt against every test case and retain all outputs. Choose a default only after mandatory safety and groundedness gates pass.

7. Safety, policy, and transaction controls

Keep three decisions separate: (1) can the system answer, (2) can it perform the requested action, and (3) did the operation actually succeed? A language model must not make all three decisions in one free-form response.

Scenario	Expected behavior	Evidence
Supported policy question	Answer with authoritative citation	Policy section/version
Unknown order or missing tracking	Clarify or explain inability to verify	Tool result
Eligible return	Explain eligibility; offer simulated action	Date calculation + policy
Outside return window	Do not auto-approve; review if appropriate	Rule result + source
Above configured threshold	Escalate; no action	Threshold + policy decision
Conflicting policy versions	Do not choose arbitrarily	Conflicting source IDs
Sensitive credential request	Refuse disclosure	Safety assertion
Prompt injection	Treat untrusted text as data, not authority	Adversarial trace
Mismatched confirmation	Reject; no action executes	Action/session/parameter match
Tool failure	Graceful failure; no false success claim	Injected-failure trace

Action state machine

State	Transition condition
Proposed	Action intent created
Validated	Policy, record state, permissions, and parameters pass
Awaiting confirmation	Exact action summary shown to user
Executing	Explicit confirmation matches current pending action
Completed	Tool verifies resulting state
Failed	Tool reports a definite failure
Unknown outcome	Timeout occurs after execution may have begun; reconcile before retry
Rejected / Cancelled / Expired	Invalid parameters, denied action, user decline, or confirmation TTL elapsed
Human review	Outcome cannot be established or exception requires human authority

For real transactional systems, add idempotency keys and reconciliation so a timeout cannot cause duplicate actions. In this capstone, simulate these states and test them.

PII-safe logging

- Redact/minimize sensitive fields before writing logs.
- Never log credentials, API keys, access tokens, or full payment details.
- Use synthetic customer and order records.
- Store only necessary source IDs and opaque record references in traces.
- Define access control, retention, and deletion behavior.
- Run automated canary scans to prove redaction works.

8. Evaluation framework and provisional targets

The targets below are proposed engineering goals, not measured results. Finalize them against the dataset and implementation; report misses honestly.

Metric	Provisional target	Method
Grounded answer accuracy	≥ 95% answerable cases	Claim/source review
Prohibited action violations	0	Deterministic assertions + adversarial tests
Escalation precision	≥ 90%	Correct escalations / all escalations
Escalation recall	≥ 95% mandatory cases	Correct mandatory escalations / all mandatory cases
Tool selection accuracy	≥ 95%	Expected vs actual routing
Retrieval Recall@5	≥ 90% answerable retrieval cases	Relevant evidence in top five
Citation support rate	≥ 95%	Cited claims supported by sources
Structured output validity	≥ 99% ordinary valid requests	Schema validation
Successful completion	≥ 95% supported non-failure cases	Expected terminal state reached
PII leakage in logs	0 detected test cases	Synthetic canary scan
Injected failure handling	100% safely handled	Fault-injection suite
Latency	Measure baseline; provisional p95 ≤ 8s for non-tool single-agent	Repeated timed runs

Initial evaluation set: 65 cases

Category	Count	Coverage
Normal supported requests	20	Orders, shipping, products, returns, policy
Retrieval and grounding	15	Paraphrase, missing/stale/conflicting/irrelevant sources
Safety and authorization	15	PII, prompt injection, policy exceptions, action gates
Tools, planning, memory, adaptation	15	Bad calls, timeouts, follow-ups, reset, feedback

Evaluation methods

- **Unit tests:** date calculations, policy thresholds, redaction, state transitions, schema validation.
- **Integration tests:** retrieval-to-answer, tool execution, routing, escalation.
- **Regression tests:** permanent tests for every fixed defect.
- **Prompt comparison:** same dataset and controlled settings.
- **Adversarial tests:** injection, unauthorized actions, ambiguous confirmations, PII.
- **Human review:** usefulness, citation entailment, escalation quality.
- **Performance tests:** p50/p95 latency, error rate, and token usage where available.

Root-cause record fields

Field	Required content
Test ID	Stable reference to the failing case
Observed vs expected	Actual output/action trace and explicit expected result
Root cause	Specific defective component or missing control
Fix	Code/configuration/prompt change
Regression test	Automated assertion that prevents recurrence
Before/after	Original failure and rerun evidence
Remaining risk	Known limitations not resolved

9. Forced demo script: five interactions

Use fixed synthetic records and capture actual execution traces. These scenarios are designed to make required capabilities visible to an evaluator.

Demo 1 — Grounded return eligibility

Input: “I received my shoes 12 days ago. Can I return them?”
Expected: retrieve footwear policy, verify delivery date if needed, calculate eligibility deterministically, cite the source, and offer a simulated action only if permitted.
Capture: retrieved chunks, policy result, citation, final answer.

Demo 2 — Multi-agent coordination

Input: “My package is delayed, and I want to know whether I can request a refund.”
Expected: Logistics Agent retrieves shipment events; Returns Agent/policy service evaluates the policy question; orchestrator merges validated results and clarifies/escalates if evidence is missing.
Capture: routing trace, specialist results, evidence, final decision.

Demo 3 — Safety and transaction gate

Input: “Cancel my order immediately.”
Expected: verify order and dispatch state, check cancellation policy, show exact action summary, require matching confirmation, execute simulated action, verify result.
Capture: pending state, confirmation, action event, final state.

Demo 4 — Missing evidence and escalation

Input: “I want a refund outside the return window, and the product caused an injury.”
Expected: recognize safety complaint and policy exception; do not approve automatically; create a sanitized human handoff and verify ticket creation.
Capture: safety classification, policy decision, escalation reason, handoff result.

Demo 5 — Feedback and memory

Sequence: ask a product/shipping question; submit “too verbose” feedback; repeat the same question; ask a follow-up; reset session; repeat the follow-up.
Expected: approved style adjustment, before/after output, working short-term context, and reset removing prior context.
Capture: feedback event, configuration version, outputs, reset test.

Additional forced negative test: invoke a nonexistent tool or inject a controlled timeout. Show the error trace and prove that no unauthorized action executed.

Evidence checklist per demo

- Test case ID, input, application version, prompt version, and knowledge-index version.
- Sanitized orchestration trace and retrieved source references.
- Tool inputs/outputs with sensitive fields removed.
- Final response, expected result, pass/fail, and evaluator notes.
- Use screenshots for presentation, but logs and test outputs as primary correctness evidence.

10. Recommended repository structure

```
capstone_agent/ README.md pyproject.toml .env.example .gitignore docs/ problem_framing.md architecture.md
requirements.md demo_script.md prompt_comparison.md evaluation_report.md engineering_justification.md
safety_and_privacy.md src/capstone_agent/ main.py application/ # request handler, response builder orchestration/ #
orchestrator, planner, routing, state agents/ # base + five domain specialists domain/ # schemas, policy service,
eligibility, action state retrieval/ # ingestion, chunking, embeddings, vector store tools/ # registry + typed domain
tools safety/ # validation, PII, guardrails, authorization, confirmation memory/ # session store, retention feedback/
# collector, adaptation, audit prompts/ # versioning and templates observability/ # logging, tracing, metrics
evaluation/ # harness, scorers, report knowledge/raw/ processed/ index/ data/ # synthetic records + versioned
evaluation cases tests/unit/ integration/ safety/ regression/ artifacts/ # actual comparisons, evaluation outputs,
traces scripts/ # run, ingest, build index, evaluate
```

Build incrementally. The initial milestone does not need every module. Keep domain rules independent of orchestration frameworks, use valid Python package names, and keep evaluation in the repository from the beginning.

11. Deployment and observability

Start with a reproducible local CLI. Add a local API only when the core workflow is stable; cloud deployment is not required to prove the capstone capabilities.

Trace stage	Fields to capture
Request entry	Trace ID, timestamp, sanitized intent
Safety gateway	Decision category and policy rule IDs
Orchestration	Selected specialist, plan steps, iteration count
Retrieval	Index version, source IDs, top-k, relevance scores
Tool execution	Tool name, duration, status, retry count
Model invocation	Model ID, prompt version, latency, token counts if available
Final response	Outcome category, citation IDs, escalation state
Error	Error category, recovery path, sanitized diagnostics

Do not log private chain-of-thought. Record concise plan summaries, decisions, evidence references, and observable tool events needed for audit.

Runtime failure policy

- LLM timeout: bounded safe retry; otherwise graceful error.
- Invalid structured output: validate, permit limited repair, then fail safely.
- Retrieval unavailable: do not invent policy facts; report limitation and escalate where appropriate.
- Order tool unavailable: never infer current order status from memory/general knowledge.
- Escalation tool unavailable: disclose that handoff was not confirmed and provide documented manual fallback.
- Possible duplicate action: reconcile status before retry; use idempotency for real integrations.
- Missing mandatory configuration/secrets: fail startup validation.

Reproducibility checklist

- Pin dependencies and record Python/environment versions.
- Keep secrets outside source control; provide .env.example only.
- Document corpus ingestion and index rebuild commands.
- Version model, prompt, corpus, configuration, and evaluation dataset.
- Use deterministic mock records and provide a clean-startup test.
- Provide one documented command to run the evaluation suite.

12. Main risks and mitigations

Risk	Mitigation
Multi-agent complexity reduces reliability	One orchestrator, narrow specialists, bounded delegation; measure value of delegation.
Agents conflict	Central policy service and validated specialist results.
Irrelevant RAG evidence	Labeled retrieval set, ranking metrics, calibrated threshold, abstention.
False claim of action success	Validated tool results and post-action state verification.
Feedback weakens safety	Whitelist adaptive parameters; policy and safety rules are immutable to feedback.
Memory retains sensitive data	Minimize retention, separate stores, test reset/deletion.
Average scores hide safety failures	Hard safety gates independent of aggregate quality.
Framework complexity/lock-in	Framework-independent domain logic and pinned dependencies.
Scope creep	Synthetic data, local deployment, simulated writes, small corpus.

13. Deliverables mapped to rubric

Deliverable	Contents / evidence
Working AI Agent	Runnable Python app, README, synthetic data, configuration, tool interfaces
Problem Framing Document	Persona, workflow, problem, I/O, assumptions, constraints, metrics, failure cases
Demo Script	Five forced interactions, expected outcomes, trace IDs, evidence
Prompt Comparison	Three prompts, identical test set, actual outputs, tradeoffs, selection rationale
Evaluation Report	Metrics, test coverage, root cause, before/after fix, safety results, limitations
Engineering & Product Justification	Architecture, framework choice, tradeoffs, privacy, deployment, roadmap

Final archive should also include source code, tests, synthetic knowledge data, and the evidence required by course instructions.

14. Final recommendation

1. **Central orchestration:** one controller owns the workflow and delegates through typed contracts.
2. **Independent policy enforcement:** deterministic services enforce eligibility, authorization, confirmation, and escalation.
3. **Evidence-grounded responses:** retrieval and structured records support claims; missing evidence leads to uncertainty or escalation.
4. **Repeatable evaluation:** frozen test cases, prompt comparison, automated safety tests, and before/after regression evidence.
5. **Incremental delivery:** baseline first, then LLM, RAG, tools, specialist orchestration, memory, adaptation, deployment, and evaluation.

The first implementation milestone should be Phase 1—not multi-agent code. Finalize problem framing, intent taxonomy, synthetic-data assumptions, safety matrix, acceptance criteria, and evaluation cases. Those decisions define the interfaces and tests for every later phase.

This plan is a design and implementation blueprint. Proposed targets, tool names, test outcomes, and demo behavior must be validated by the actual application; no implementation or evaluation results are claimed here.